



Estruturas de Repetição

for

Linguagem de Programação (Python)

Por que usar repetição?

Sem repetição

```
print(1)  
print(2)  
print(3)  
print(4)  
print(5)
```

Com for

```
for numero in range(1, 6):  
    print(numero)
```

Ideia principal

Uma instrução pode ser executada várias vezes de forma controlada.

Estrutura básica do for



```
for variável in sequência:  
    comando
```

- for inicia a repetição
- a variável recebe um valor a cada repetição
- in indica a sequência percorrida
- os comandos indentados pertencem ao for



range(): controlando a repetição

```
for numero in range(1, 6):  
    print(numero)
```

Resultado

1
2
3
4
5

Regra

O valor final do range() não é incluído.

range(1, 6) → 1 até 5

Início, fim e passo

```
range(início, fim, passo)
```

```
for numero in range(2, 11, 2):  
    print(numero)
```

Resultado

2, 4, 6, 8, 10

Lembre-se

início é incluído
fim não é incluído
passo define o intervalo

A variável do for

```
for contador in range(1, 6):  
    print(contador)
```

- 1ª repetição → contador = 1
- 2ª repetição → contador = 2
- 3ª repetição → contador = 3
- ...
- A variável muda de valor a cada repetição.

for com cálculos

```
for numero in range(1, 6):  
    dobro = numero * 2  
    print(dobro)
```

A cada repetição

1 → 2
2 → 4
3 → 6
4 → 8
5 → 10

Perceba

O cálculo acontece novamente para cada valor de numero.

for + if

Repetição e decisão podem trabalhar juntas.

```
for numero in range(1, 11):  
    if numero % 2 == 0:  
        print(numero)
```

O que acontece?

1. O for percorre os números.
2. O if verifica a condição.
3. Apenas os pares são exibidos.

Acumulador

```
soma = 0

for numero in range(1, 6):
    soma = soma + numero

print(soma)
```

Como funciona

soma começa em 0 e é atualizada a cada repetição.

Conceito

Acumulador → guarda um valor que vai sendo atualizado.

Dois cuidados importantes



1. Indentação

```
for numero in range(1, 6):  
    print(numero)
```

O print precisa estar indentado.

2. Limite do range()

```
range(1, 5)  
→ 1, 2, 3, 4
```

O 5 não é incluído.



Como pensar antes de programar?



1

O que precisa ser repetido?

2

Quantas vezes?

3

Qual variável controla a repetição?

- Se a quantidade de repetições é conhecida, o for costuma ser uma boa escolha.
- Primeiro analise o problema; depois escreva o código.



Atividade de fixação 1

Contagem regressiva

- Mostre uma contagem regressiva de 10 até 1.
- Ao final, exiba: “Fim da contagem!”.
- Identifique a variável e o tipo de dado antes de programar.

Atividade de fixação 2

Números múltiplos de 3

- Mostre todos os múltiplos de 3 entre 1 e 30.
- Pense em como o `range()` pode ajudar.
- Use uma condição para selecionar os valores.

Atividade de fixação 3

Cálculo de notas

- Solicite 4 notas, uma por vez.
- Ao final, calcule e mostre a média.
- Defina as variáveis antes de escrever o código.

Atividade de fixação 4

Cadastro de produtos

- Solicite o nome de 5 produtos.
- A cada produto, mostre uma confirmação de cadastro.
- Ao final, informe que todos foram cadastrados.